

Towards Hybrid Architectures for Big Data Analytics: Insights From Spark-MPI Integration

Mengbing Zhou , Qiuyan Li, Mingyuan Cai, Chengzhong Xu, *Fellow, IEEE*, and Yang Wang , *Member, IEEE*

(Survey-Tutorial Paper)

Abstract—High-Performance Data Analytics (HPDA) combines high-performance computing (HPC) with data analytics to uncover patterns and insights in dual-intensive applications that are both data-intensive and compute-intensive. Traditional Big Data frameworks and HPC technologies often struggle to address these demands independently, prompting researchers to explore their integration. Spark, known for its efficient in-memory computing with RDDs, and MPI, a foundational standard in HPC, are prominent candidates for such integration. This survey explores the integration of Spark and MPI for HPDA, highlighting their potential for unified data processing and computation. We first classify application workloads and review the characteristics and limitations of traditional frameworks. Then, we analyze the challenges and requirements of integrated architectures, focusing on the specific implementations of typical middleware-level architectures. Through comparative analysis, we highlight their advantages and limitations. Finally, we present application examples, outline key challenges and future research directions, and briefly discuss progress in integration approaches for other technology combinations.

Index Terms—Spark, MPI, big data, high-performance computing, dual-intensive application, integration technology.

I. INTRODUCTION

Artificial intelligence technologies continue to develop, more and more applications require both data-intensive and compute-intensive processing [1], [2], [3]. In the realm of classic supercomputing applications, such as weather forecasting [4], [5], energy exploration and exploitation [6], as

well as satellite remote sensing technology [7], the data scale involved is rapidly expanding alongside the widening scope of monitoring. Similarly, emerging supercomputing domains like autonomous driving technology [8], [9], genetic sequencing research [10], and neuromorphic science simulations [11], the majority of data volumes have reached the petabyte level, highlighting a prominent characteristic of data intensity. These evolving trends collectively pose unprecedented challenges to high-performance computing (HPC) storage systems: they must not only support larger volumes of data, handle a broader variety of data formats, but also efficiently manage the execution of numerous parallel tasks while ensuring high data reliability [12], [13]. Consequently, the evolution of HPC storage systems must focus on delivering broader bandwidth capabilities, higher input/output operations per second (IOPS), rock-solid stability, and the capacity to accommodate massive concurrent access demands. Moreover, many of these applications require fast response times to meet performance demands. For example, training deep learning models involves intensive I/O operations and tensor calculations to ensure accuracy [14], [15]. If the training process is excessively time-consuming, it may delay deployment and hinder timely model updates [16]. Hence, dual-intensive applications require the support of both Big Data and HPC technologies to ensure efficient processing.

Currently, typical Big Data frameworks are designed primarily for large-scale data processing, prioritizing high-bandwidth I/O operations for independent tasks. However, they offer limited support for HPC workloads. Conversely, traditional HPC clusters excel at scientific workflows, emphasizing low-latency I/O operations for inter-task communication, but lack efficient mechanisms for handling high-bandwidth Big Data processing [17]. As a result, neither framework type can fully meet the demands of applications requiring both intensive computation and data handling [18]. This gap underscores the need for integrating Big Data frameworks with HPC technologies. Among various approaches, the combination of Spark [19] and MPI (Message Passing Interface) [20] stands out, not only for its significant research attention but also because of its complementary strengths in addressing the diverse demands of dual-intensive workloads.

Spark, a widely used Big Data framework in large data centers, excels in handling data-intensive tasks. By leveraging in-memory computing, Spark reduces latency and enhances performance for iterative algorithms and interactive queries, making it highly efficient for workloads that involve substantial data shuffling and transformation. However, for compute-intensive tasks such as the k-means clustering algorithm [21],

Received 12 January 2025; revised 31 March 2025; accepted 2 April 2025. Date of publication 18 April 2025; date of current version 12 June 2025. This work was supported in part by the 3rd Xinjiang Scientific Expedition Program under Grant 2021xjkk1300, in part by Shenzhen Science and Technology Plan Project, Shenzhen-Hong Kong-Macau Category C, under Grant SGDX20220530111001003, and in part by Shenzhen Science and Technology Program under Grant CJGJZD20230724093659004. (Mengbing Zhou and Qiuyan Li contributed equally to this work.) (Corresponding author: Yang Wang.)

Mengbing Zhou and Mingyuan Cai are with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, and also with the University of Chinese Academy of Sciences, Beijing 100049, China.

Qiuyan Li is with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China.

Chengzhong Xu is with the Faculty of Science and Technology, University of Macau, Macau, China.

Yang Wang is with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen 518055, China, also with the University of Chinese Academy of Sciences, Beijing 100049, China, and also with the Shenzhen University of Advanced Technology, Shenzhen 518107, China (e-mail: yang.wang1@siat.ac.cn).

Digital Object Identifier 10.1109/TSC.2025.3562342

graph analysis [22], large matrix factorizations [23], its performance falls short compared to traditional HPC technologies like MPI, which are optimized for low-level data distribution and communication, providing superior efficiency for these types of computations. Gittens et al. [24] compared the performance of Spark and MPI in matrix factorization problems through experiments. Their results showed that Spark performed $10\times$ to $40\times$ worse than MPI on the Cori system at NERSC. Similarly, the experimental results of Reyes-Ortiz et al. [25] demonstrated that MPI outperforms Spark by more than $10\times$ in processing speed. Saxena et al. [26] extended this comparison to Python-based data science applications, focusing on natural language processing (NLP) tasks. Their findings also showed that MPI offered better performance compared to PySpark. Despite its superior performance in compute-intensive tasks, MPI lacks integrated mechanisms for Big Data storage and management, as well as fault tolerance capabilities, making it less suitable for data-intensive workloads.

Therefore, the integration of Big Data frameworks and HPC, which not only retains the Big Data processing mechanism of Big Data framework but also achieves the performance of HPC technology, has become a promising approach. For example, Qiu et al. [27] proposed a preliminary implementation aimed at integrating HPC and Big Data systems, which is called the "High-Performance Computing-Big Data Stack" (HPC-ABDS) system. The authors hope to provide complete and efficient support for dual-intensive application workloads through the integration architecture [28]. Due to varying application environments and objectives, the integration of Big Data and HPC is still evolving, with related technologies yet to reach maturity. Numerous challenges remain to be addressed urgently to achieve seamless integration [29].

Given this context, this survey presents some typical integration architectures of Spark and MPI, each with a different technical focus, and provides a comparative analysis. It also introduces some new optimization ideas, hoping to help promote the further improvement of the integration architecture and make it more widely used. Through this survey, readers can gain valuable insights into selecting and optimizing integration architectures for scenarios that are both data- and compute-intensive.

The remainder of this survey is organized as follows: Section II provides an overview of workload classification and the specific requirements for dual-intensive workloads. Section III analyzes the characteristics and limitations of Spark and MPI. Section IV explores the challenges of integration and proposes potential strategies. Section V highlights several representative integration architectures, followed by practical application examples in Section VI. Section VII presents an open discussion on unresolved challenges. Section VIII briefly examines advancements in alternative combinatorial approaches, and finally, the survey is concluded in Section IX.

II. APPLICATIONS

In the field of parallel and distributed computing, applications exhibit varying demands for data processing and computing power. The nature of the workload determines the specific requirements for the computing framework.

A. Classification of Workloads

We categorize workloads into data-intensive, compute-intensive, and dual-intensive types, analyzing their

characteristics and identifying suitable computational frameworks for each.

- *Data-Intensive Workloads:* These workloads involve processing, storing, and analyzing large volumes of data. Their performance is often limited by data movement and storage overhead, requiring efficient data management. Examples include large-scale data analytics, such as log analysis [30], ETL (Extract, Transform, Load) processes [31], and data cleansing [32]. They also include data replication and backup [33], large-scale data migration [34], and real-time stream processing [35]. In addition, data-driven machine learning tasks involving large datasets [36] are typically categorized as data-intensive. Big data frameworks are well-suited for these scenarios, providing distributed and scalable data processing capabilities.
- *Compute-Intensive Workloads:* Compute-intensive tasks demand significant computational power for complex calculations, often in scientific simulations or numerical analysis. These workloads are typically CPU- or GPU-bound. Examples include weather forecasting [37], [38], molecular dynamics [39], and physical simulations [40], which involve intensive floating-point operations. More generally, they encompass scientific computations such as matrix operations [41] and large-scale parallel computing tasks [42], as well as graph algorithms like PageRank [43] and community detection [44]. MPI is the preferred framework here, providing high performance and scalability for parallel computing.
- *Dual-Intensive Workloads:* These workloads require both extensive data handling and intensive computation, combining characteristics of the other two types. Examples include large-scale deep learning [45], [46], high-dimensional data analysis [47], [48], and genomics analysis involving sequence alignment [49] and variant analysis [50]. These tasks demand a balance between data throughput and computational efficiency, which neither Big Data frameworks nor MPI can achieve alone. Hybrid approaches are essential for meeting their performance requirements.

While these categories offer a useful framework for understanding workloads, the boundaries between data-intensive and compute-intensive tasks are becoming increasingly blurred. Modern workloads often exhibit characteristics of both, evolving into dual-intensive tasks that require both extensive data processing and significant computation. This shift underscores the need for hybrid architectures that integrate Big Data frameworks with HPC technologies, enabling efficient handling of these complex and resource-intensive scenarios.

B. Dual-Intensive Workload Requirements

Dual-intensive workloads require a seamless integration of Big Data frameworks' data-handling capabilities with HPC's computational efficiency. Achieving this balance is essential for effective processing and computation in distributed environments.

Big data frameworks are highly effective at managing and processing vast datasets across distributed systems. Besides data recovery, efficient data transfer is crucial for performance and scalability, as it directly impacts system efficiency and stability. The bandwidth requirements of these frameworks depend on factors such as data volume, the number of compute nodes, and replication or partitioning strategies. To maintain smooth

operations in large-scale data processing, high throughput is critical. Modern data centers often employ Fibre Channel [51] as a dedicated storage networking solution, ensuring high-bandwidth access to large datasets.

Conversely, HPC technologies, such as those built on MPI, are tailored for computationally intensive workloads. These typically involve complex numerical calculations and simulations that demand substantial processing power and efficient resource allocation. HPC systems optimize performance by employing parallel processing across multiple nodes, ensuring low-latency communication and effective synchronization. MPI, in particular, is well-suited for such workloads due to its capability to facilitate high-speed inter-node communication and support sophisticated computational tasks. Additionally, InfiniBand-based RDMA (Remote Direct Memory Access) transmission [52], [53] is widely used in HPC environments.

To effectively address the challenges of dual-intensive workloads, systems must meet the following key performance requirements:

- *Computational Power*: The system must deliver substantial processing capabilities to handle complex calculations and simulations across distributed nodes efficiently.
- *Storage Performance*: A high-performance storage subsystem is necessary to support rapid data read/write operations and low-latency access, ensuring timely processing of large datasets.
- *Network Efficiency*: High-speed, low-latency network infrastructure is critical to support large-scale parallel computing and enable seamless synchronization and data exchange among nodes.
- *Resilience*: The system must ensure fault tolerance, preserving data integrity and computational reliability, as both distributed processing and HPC environments are prone to failures and disruptions.
- *Scalability*: The system must adapt to both small-scale and large-scale workloads, maintaining high performance and reliability under varying demands.

These requirements highlight the limitations of using Big Data frameworks or HPC systems in isolation. Big data frameworks excel in managing and analyzing large datasets, while HPC systems are optimized for computationally intensive tasks. However, neither can fully address the combined demands of dual-intensive workloads. A hybrid approach that integrates the strengths of both—leveraging Big Data frameworks for efficient data handling and HPC technologies for computational optimization—offers a balanced solution, ensuring effective and high-performance processing of dual-intensive tasks.

III. TRADITIONAL FRAMEWORK

Traditionally, computational frameworks have been categorized into Big Data processing and high-performance computing frameworks based on workload types and requirements. Below, we introduce their representative tools: Spark for Big Data and MPI for HPC.

A. Apache Spark

Spark [19] is a general-purpose in-memory parallel computing framework developed by the AMP Lab at the University of California, Berkeley. It offers several advantages over previous Big Data frameworks, such as MapReduce [54]. One key feature

is that Resilient Distributed Datasets (RDDs) can be stored in memory, significantly improving processing speed. Additionally, Spark utilizes a Directed Acyclic Graph (DAG) for global optimization, which enables better performance compared to the two-stage programming model of Map and Reduce. Spark's operators also offer more flexible expressions, providing greater programming versatility.

Spark can be characterized by two key aspects: data access, centered around RDDs and their persistence mechanisms for efficient in-memory storage, and execution, which leverages DAGs and lazy evaluation to optimize performance. Additionally, its coding style and limitations are also discussed.

1) *Resilient Distributed Dataset and Persistence Mechanism*: RDDs form the core abstraction in Spark, providing a fault-tolerant, read-only collection of objects that can be processed in parallel. RDDs can be created from a distributed file system, such as HDFS [55], or derived from an existing RDD through transformations. They are designed with five key attributes that enable efficient data partitioning, parallel task scheduling, and fault tolerance:

- *List of partitions*: Spark logically divides a large dataset into smaller subsets called partitions for parallel processing. Each partition is processed by a node, and the partition list maintains metadata about these subsets.
- *Partitioner*: A partitioner determines how data is distributed among partitions. Spark supports two partitioning strategies: *HashPartitioner* and *RangePartitioner*.
- *Function for computing each split*: This defines the logic for processing each partition, specifying how data within a partition is computed or transformed.
- *List of dependencies*: Transformations on RDDs are tracked as dependencies, forming a lineage that ensures fault tolerance. If a partition is lost, Spark can recompute it based on its lineage.
- *Preferred locations*: This attribute specifies the optimal physical locations for processing each partition, following the Compute Near Storage policy to minimize data transfer overhead.

RDDs provide Spark with several significant advantages. First, they enable efficient parallel execution by dividing work among nodes. Second, correct partitioning minimizes network communication between nodes, adhering to the principle of data locality and improving overall performance.

To further optimize execution, Spark provides a flexible persistence mechanism, allowing users to explicitly persist an RDD in memory, on disk, or a combination of both. When an RDD is persisted, its partitions are stored on their respective compute nodes. If available memory is insufficient to hold all partitions, Spark either spills the excess to disk or recomputes them as needed during subsequent operations.

This persistence mechanism is particularly beneficial for iterative computations. Many machine learning algorithms require multiple passes over the same dataset to optimize parameters. By caching the RDD in memory, Spark eliminates repeated disk reads, a limitation of frameworks like MapReduce, thereby reducing I/O overhead and accelerating computation.

2) *Directed Acyclic Graph and Lazy Evaluation*: Two critical concepts in Spark are the DAG and lazy evaluation, both of which play a pivotal role in improving performance.

A DAG represents the dependencies between RDDs. These dependencies are defined by operators and are classified into *narrow dependencies* and *wide dependencies*. Narrow

TABLE I
MPI BASIC INTERFACE

Interface	Description
MPI_INIT()	Initializes the MPI environment.
MPI_FINALIZE()	Terminates the MPI environment.
MPI_COMM_RANK(comm, rank)	Determines the rank of the calling process in the communicator.
MPI_COMM_SIZE(comm, size)	Determines the number of processes in the communicator.
MPI_SEND(buf, count, datatype, dest, tag, comm)	Sends a message to a specified process.
MPI_RECV(buf, count, datatype, source, tag, comm, status)	Receives a message from a specified process.

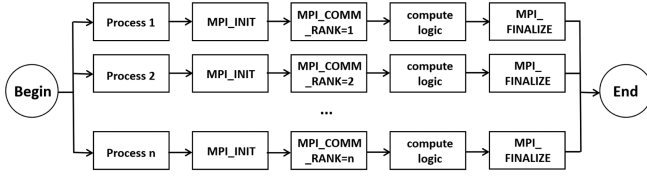


Fig. 3. General program flow of MPI.

scalability and performance over PySpark. These results underscore Spark's challenges in compute-intensive and iterative scenarios, emphasizing the need to carefully evaluate task-specific requirements when selecting computing frameworks.

B. Message Passing Interface (MPI)

MPI [20] is a standard and programming model specifically designed to enable high-performance communication between processes. It employs a messaging-based paradigm where processes exchange data through explicit message passing, providing a highly efficient mechanism for parallel computation. MPI's design ensures compatibility and portability, allowing it to function seamlessly across heterogeneous cluster environments and a wide range of system architectures. Moreover, its communication model leverages advanced inter-process communication techniques to minimize latency and maximize throughput, making it a robust choice for HPC applications.

1) *Programming Pattern*: OpenMPI [56] and MPICH [57] are two widely used implementations of MPI, providing C and FORTRAN interfaces to the MPI library. The MPI library encompasses a comprehensive set of interfaces, among which the six listed in Table I are fundamental and form a complete subset of MPI functionality. These core interfaces are essential for implementing basic message-passing operations and serve as the building blocks for more complex parallel applications.

The basic structure of an MPI program is shown in Fig. 3, highlighting its flexibility in supporting different programming patterns. MPI parallel programming can be classified into two main modes: peer-to-peer and main. In peer-to-peer mode, all processes are equal in status and functionality, executing the same code but operating on different subsets of data or tasks. This mode is particularly suitable for applications that require distributed yet uniform task execution, such as numerical simulations.

In contrast, the main-secondary mode involves a distinct division of labor. A main process is responsible for coordinating the overall computation and distributing tasks, while one or more secondary processes execute the assigned tasks. This mode divides the MPI program into two distinct code paths: one for

the main process and another for the secondary processes. It is especially effective in scenarios where task scheduling and central control are required, such as dynamic workload distribution or iterative processes.

This flexibility in programming patterns enables MPI to cater to a wide variety of parallel computing applications, ensuring both scalability and adaptability in diverse computational scenarios.

2) *Communication Pattern*: MPI supports four primary communication modes: standard mode, buffered mode, synchronous mode, and ready mode. These modes differ based on whether the data being sent is cached and whether the sending process takes the state of the receiving process into account. Each mode is designed to cater to specific communication requirements. For instance, the standard mode allows basic data exchange, while the buffered mode offers more flexibility by enabling the sending process to proceed without waiting for the receiver to be ready. Synchronous mode ensures that the sender waits for an acknowledgment from the receiver, providing a guarantee that the message has been received before proceeding. Ready mode, on the other hand, requires the receiver to be ready before the sender transmits data.

A crucial feature in MPI is non-blocking communication, which improves performance by allowing processes to continue executing other tasks while data is being transferred. Non-blocking operations can be combined with the aforementioned modes to create new communication patterns that further enhance efficiency and reduce idle times. The variety of communication modes in MPI contributes significantly to its high communication performance, providing the flexibility to tailor the communication strategy to the specific needs of a parallel application.

In addition to these point-to-point modes, MPI also supports group communication. Unlike point-to-point communication, which involves two processes, group communication allows multiple processes to participate simultaneously in data exchange. This mode is especially valuable for parallel computations that require synchronization among a set of processes or involve collective data operations. Group communication is commonly used in scenarios like parallel reductions, broadcasts, and collective data synchronization, which are essential in many scientific and engineering applications.

3) *Limitation*: Despite its strengths in managing compute-intensive tasks through an efficient communication model, MPI has several notable limitations. One significant challenge is the complexity of programming and debugging. MPI requires explicit management of communication between processes, which can result in intricate, error-prone code, especially when dealing with complex parallel algorithms. This steep learning curve

presents a barrier for many developers, as it demands a thorough understanding of parallel computing concepts and the specifics of MPI's communication mechanisms.

Another major drawback of MPI is its lack of built-in fault tolerance. Unlike frameworks such as Spark, which offer resilience features like data replication and automatic recovery, MPI does not inherently provide mechanisms to handle node failures or network issues. As a result, developers are required to implement their own fault tolerance strategies, making MPI more difficult to use in environments where reliability is critical.

Furthermore, MPI's efficiency can degrade when applied to data-intensive tasks. Unlike Spark, which leverages in-memory data processing with resilient data structures to accelerate computations, MPI lacks similar high-level abstractions and typically relies on more manual management of data access, often using traditional disk-based storage. This can become a performance bottleneck, especially in applications where large datasets need to be handled. As a result, MPI may not be the most suitable choice for tasks that require both high computational power and efficient management of large volumes of data. These limitations reduce MPI's effectiveness in scenarios that require robust fault tolerance, scalability for large datasets, and in-memory computation, which are strengths of frameworks like Spark.

IV. INTEGRATED ARCHITECTURE COMMONALITY

As dual-intensive applications expand, the need to integrate Big Data frameworks with HPC technologies grows. This integration seeks to combine the scalability and data-handling capabilities of Big Data frameworks, such as Spark, with the computational power of HPC technologies like MPI. However, achieving seamless integration is challenging due to the fundamental architectural differences and distinct optimization strategies between these technologies.

A. Technical Difficulties

Integrating Big Data frameworks with HPC technologies presents significant challenges. These challenges arise from the fundamental differences between Big Data and HPC, both at the architectural and application layers, making seamless integration difficult.

1) *Architectural Layer*: The primary goal of Big Data technology is to efficiently process large volumes of data. To handle such massive datasets, distributed architectures are commonly employed in Big Data frameworks. These architectures are typically divided into two components: distributed computing and distributed storage. Traditional Big Data frameworks often adopt a coupled architecture for storage and computing [58], where both are located on the same cluster node. This setup aims to minimize performance losses caused by data transfer between nodes by leveraging a compute-near-storage strategy.

In contrast, the goal of HPC centers is to address the performance challenges of complex computations [59]. A typical HPC cluster consists of computing systems, storage systems, and high-speed networking systems, where nodes communicate over the network and access data via the storage system. HPC clusters usually adopt a decoupled architecture for computing and storage, where compute nodes do not typically concern themselves with the physical location of data, thanks to the abstraction provided by distributed file systems.

The architectural differences between Big Data frameworks and HPC centers [60], [61] result in distinct optimization strategies. Big data frameworks emphasize data locality to minimize data movement, whereas HPC clusters focus on load balancing to maximize overall performance. However, prioritizing one may sometimes come at the expense of the other. Consequently, integrating Big Data frameworks with HPC centers requires carefully balancing these trade-offs and harmonizing the optimization strategies of both systems.

2) *Application Layer*: High-performance computing is primarily used to solve computationally intensive problems, such as computational fluid dynamics [62], astrophysics [63], and various numerical simulations. As a result, the application data formats in HPC are highly diverse [64]. For instance, remote sensing applications often use image formats [65], with standardized storage formats such as GeoTIFF [66], HDF [67], and netCDF [68] being common in satellite imaging. In contrast, the core data structure in Spark is the RDD. HPC file systems like HDF and netCDF are not natively supported by Big Data frameworks such as Spark, which presents a challenge when attempting to integrate these systems.

The difficulty lies in the fact that not all HPC data formats can be converted into formats recognized by Big Data frameworks. HPC data formats tend to have deep hierarchical structures, making it challenging to serialize them simply as strings [69]. Additionally, for some HPC applications, the overhead incurred from converting data formats—both in terms of time and storage—becomes prohibitive when dealing with large datasets. This transformation overhead can significantly impact the performance of the integration architecture, making it an essential challenge to address when integrating HPC with Big Data frameworks.

B. Integration Approaches

Given the challenges in integrating HPC and Big Data technologies, current integration strategies can be categorized into three levels based on their system architecture: Infrastructure, Middleware, and Application.

1) *Infrastructure Level*: Integrating Big Data technologies with HPC requires addressing compatibility between the data access patterns of Big Data frameworks and high-performance storage systems [70]. HPC systems emphasize storage balance, optimizing data placement and access for high-speed computations, while Big Data frameworks prioritize throughput to handle large-scale data efficiently [71].

To enable Big Data applications to better utilize high-performance storage systems, researchers have proposed two primary approaches. The first involves *software-based optimizations*, which adapt data access patterns and network configurations to improve compatibility with high-performance storage [27], [72], [73]. The second approach focuses on *architectural adjustments*, modifying the storage structures to align with the diverse access demands of Big Data and HPC paradigms [70], [74]. These methods aim to bridge the gap between the compute-centric architecture of HPC and the data-driven architecture of Big Data frameworks.

2) *Middleware Level*: Middleware-level integration approaches are categorized into three main types: direct MPI invocation in Spark, implementing Spark mechanisms with MPI, and using a cluster resource management system. These integration architectures are illustrated in Fig. 4.

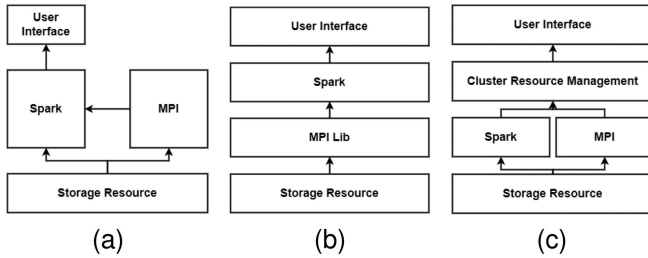


Fig. 4. Component invocation relationships in three middleware-level integration architectures. (a) Direct MPI invocation in spark. (b) Implementing spark mechanisms with MPI. (c) Using a cluster resource management system.

- *Direct MPI Invocation in Spark* [75], [76] (Fig. 4(a)): This approach typically involves designing interface functions within Spark to invoke MPI, where the Spark cluster calls the MPI cluster for specific computational tasks requiring HPC resources. This method is relatively simple and introduces only local extensions to the Spark architecture, making it easy to implement. However, the integration level remains relatively low due to the lack of deep architectural unification, and the approach introduces inter-cluster data communication and serialization overhead, which may impact performance.
- *Implementing Spark Mechanisms with MPI* [77], [78], [79] (Fig. 4(b)): This method integrates MPI with Big Data processing by replicating Spark's mechanisms within an MPI-based framework, effectively eliminating inter-cluster communication and serialization latency for improved performance. However, it requires substantial additional code, potentially introducing inconsistencies. The added abstractions may also reduce compatibility with existing MPI-based HPC applications. While enhancing efficiency, this approach increases complexity and limits interoperability with standard frameworks.
- *Using a Cluster Resource Management System* [80], [81] (Fig. 4(c)): This is a loosely coupled integration approach that employs third-party resource management systems to coordinate communication between Spark and MPI clusters. By leveraging global cluster information, this method allows Spark nodes to communicate with MPI clusters indirectly. It is simple to implement and deploy, requiring minimal changes to both Spark and MPI. However, it relies heavily on third-party resource management tools, which can introduce additional dependencies and limit system flexibility.

3) *Application Level*: At the application level, integration often involves handling HPC workloads using parallel programming models and Big Data workloads through data transformation frameworks. This dual approach requires additional code to coordinate execution and facilitate data exchange, increasing complexity in deployment and operations.

To address these challenges, a unified programming model has been proposed, offering a high-performance data analytics (HPDA) interface that abstracts the complexities of integrating HPC and Big Data frameworks [82]. This model simplifies development and streamlines programming, reducing the operational burden. However, it may not fully utilize hardware and software optimizations available at lower levels, potentially leading to suboptimal resource usage.

4) *Comparison*: Each of the three integration levels—Infrastructure, Middleware, and Application—offers distinct advantages and limitations, as summarized in Table II.

At the Infrastructure level, the primary benefit is the optimization of data access patterns and storage systems, enhancing compatibility between Big Data frameworks and high-performance computing. However, achieving this level of integration requires significant modifications to existing architectures, which can be resource-intensive and complex. A deep understanding of storage systems and network configurations is necessary to achieve optimal performance.

The Middleware level balances integration depth and ease of implementation. It offers varying degrees of integration tightness and enhances performance through diverse design approaches. However, this level may face challenges in maintaining compatibility and stability, especially when modifying middleware components or relying on third-party tools for resource management.

At the Application level, the focus is on providing a unified programming model that abstracts the complexities of integration, simplifying the development process and reducing deployment and operational workloads. While this approach reduces the burden on developers, it sacrifices low-level optimizations, potentially leading to less efficient resource utilization. Additionally, the need for customized code to coordinate execution and data exchange among components remains a challenge, potentially impacting scalability and maintainability.

V. TYPICAL INTEGRATION ARCHITECTURE

This section highlights middleware-level integration, showcasing its versatility through approaches like invoking MPI directly within Spark, integrating Spark mechanisms into MPI, leveraging cluster resource management systems, and employing other innovative techniques.

A. Direct MPI Invocation in Spark

To leverage both the Spark Big Data framework for comprehensive data management and the high performance of MPI parallel programs, a straightforward approach is for Spark to directly invoke MPI. In this model, Spark calls MPI to handle compute-intensive tasks during program execution.

In this integration mode, data must be transferred between the Spark cluster and the MPI cluster. Three common methods for distributed data transmission are as follows:

- *File system*: Data is stored in a distributed file system on the sender side, and the receiver reads it based on index information. This method is easy to implement but can suffer from performance degradation due to frequent file system interactions.
- *Memory as a mediator*: This method stores data in memory, such as using shared memory or distributed storage systems like Alluxio [83]. It significantly improves performance but is more complex to implement.
- *Socket*: Data is transferred directly between Spark and MPI using sockets, without intermediate media. This approach is more straightforward but may require careful handling of network communication.

The choice of data transfer method significantly shapes the integration architecture. For instance, Spark+MPI [75] and Alchemist [76], [84] adopt memory-mediated and socket-based

TABLE II
INTEGRATION LEVELS: ADVANTAGES AND DISADVANTAGES

Level	Advantages	Disadvantages
Infrastructure	Optimized data access and storage.	High complexity; major architecture changes.
Middleware	Flexible integration; diverse designs.	Compatibility and stability issues.
Application	Simplified programming; easy deployment.	Limited optimization; custom code needed.

TABLE III
SPARK+MPI PROGRAMMING INTERFACE

Interface	Explain
rddToNative(rdd, serializeFn)	Serialize the RDD as serializeFn and save it to shared memory
emptyNative(inputHandle)	Specifies the storage location for MPI calculation results
runMPI(inputHandle,outputHandle,binary, args)	Call MPI
binaryFiles(inputHandle)->partitionsIn	MPI reads data from the shared memory
savePartitions(partitionsOut, outputHandle)	Store the results to shared memory
nativeToHDFS(outputHandle)->hdfsPath	Write the results in the shared memory to the HDFS
nativeToRDD(deserializeFn,hdfsPath)->outputRDD	Generate RDD from the results in HDFS

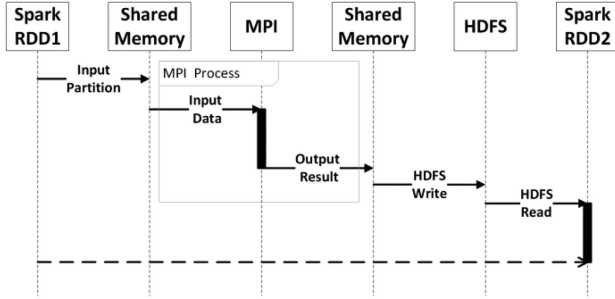


Fig. 5. Spark+MPI architecture execution process.

communication, respectively, to achieve efficient data transfer. In contrast, file systems, while easier to implement, introduce substantial latency due to storage overhead, making them less practical for dual-intensive workloads.

1) *Spark+MPI*: The Spark+MPI architecture [75] was proposed by Intel's Infrastructure Research Laboratory in 2017.

Architecture: The main design idea of this integration architecture is to use Spark's RDD data structure to store data and then call the MPI library to process the data. Data is transferred through shared memory between the Spark cluster and the MPI cluster. In this architecture, the execution flow of the algorithm is shown in Fig. 5.

First, Spark saves the initial data to the RDD. To make data available to MPI programs, Spark+MPI serializes the data and stores it in shared memory. The partition ID and hostname associated with the data are also stored in the shared memory. Second, the Spark Driver runs MPI using the standard *mpiexec* command. And then, MPI reads data from memory based on the input data address and begins to do the computing. After the calculation is complete, the results are saved to the shared memory according to the output address. And after MPI is executed, the data is stored in HDFS files for fault tolerance and persistence. MPI then exits and returns to the Spark environment. Finally, Spark can re-read data from HDFS to generate a new RDD.

Algorithm 1: Matrix-Vector Multiplications Using Spark+MPI.

Require: Matrix A , vector X
Ensure: Matrix B where $B = A \times X$

- 1: Create a Spark computing environment
- 2: Read the source data and save it to `dataRDD` %
Preparatory work before calling MPI for computation
- 3: `InputAddr` $\leftarrow$ `rddToNative(dataRDD, serializeFn)`
- 4: `OutputAddr` $\leftarrow$ `emptyNative(InputAddr)`
- 5: `runMPI(InputAddr, OutputAddr, multi_algs)` % *Switch to the MPI computing environment*
- 6: Create an MPI computing environment
- 7: `data` $\leftarrow$ `binaryFiles(InputAddr)`
- 8: Execute the computation defined in the `multi_algs` algorithm, saving the result in `result`
- 9: `savePartitions(result, OutputAddr)`
- 10: End MPI environment % *Return to the Spark computing environment*
- 11: `hdfsPath` $\leftarrow$ `nativeToHDFS(OutputAddr)`
- 12: End Spark environment

Programming Model: In order to realize the above computing flow, the Spark+MPI architecture provides many programming interfaces. The core interfaces are shown in Table III.

The Algorithm 1 is an example of running matrix-vector multiplications using the above programming interface.

First, the Spark environment is created, and the RDD matrix A and vector X are read into `dataRDD` (line 1-2). Next, interfaces are invoked to store the data in shared memory and initialize the MPI environment (line 3-5). Within the MPI environment, data is read, processed, and the results are stored in `OutputAddr` sequentially (line 6-9). Finally, the results are written from shared memory to HDFS, and the Spark environment is terminated (line 10-12).

Performance: Spark+MPI combines the advantages of Spark's Big Data framework, such as efficient data storage and

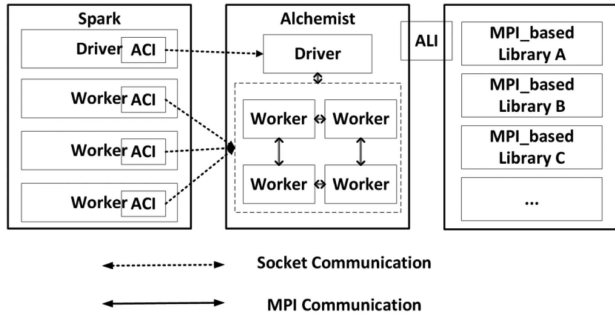


Fig. 6. Module design and interactions in alchemist architecture.

management, fault tolerance, and high-performance computing of MPI. The Spark+MPI architecture has shown good performance in several graph and machine learning applications. In a 12-node cluster, algorithms such as Latent Dirichlet Allocation (LDA), PageRank (PR), Single Source Shortest Path (SSSP), and Canonical Polyadic Decomposition (CPD) achieved performance improvements ranging from 3.1 to 17.7 times over their Spark-based implementations.

Although the Spark+MPI integrated architecture has achieved good performance improvements, there are still some shortcomings in the architecture. This architecture uses memory as the communication medium between Spark and MPI, which inevitably results in multiple copies of data. As the amount of data increases, this copying will bring huge overhead in performance, and will also cause huge pressure on memory capacity.

2) *Alchemist*: The Alchemist integration architecture [76] was proposed by the University of California, Berkeley in 2019.

Architecture: The overall idea behind Alchemist is that Spark clusters offload computationally intensive tasks to MPI clusters. The Spark cluster communicates data with the MPI cluster through sockets. Fig. 6 is the overall architecture diagram of Alchemist.

The implementation of Alchemist consists of The Alchemist-Client interface (ACI) and the Alchemist-Library interface (ALI). These two interfaces implement data communication between Spark and Alchemist and call MPI libraries respectively. These two interfaces are not only the core of Alchemist, but also the embodiment of high performance and high scalability of Alchemist.

The ACI interface is mainly responsible for network communication and data transmission between Spark and Alchemist clusters. The alchemist cluster has a main-secondary structure similar to Spark. The Driver is responsible for coordination and communication, and the Worker is responsible for calculations. Alchemist divides the data it needs to communicate into two categories. The first type of data is parameter data, which has the characteristics of a small amount of data and a single data format. Such data can be transmitted by simple serialization and communication between the Driver in Spark and Alchemist clusters. The other is distributed data, which is distributed in different nodes and has a large amount of data. The way such data is transmitted is a bright spot for Alchemist. Alchemist architecture uses a Worker node to transmit distributed data by point-to-point communication. The port information needed to establish a connection is obtained from the Driver node. Although the implementation of this method is complex, it can obtain relatively high transmission efficiency.

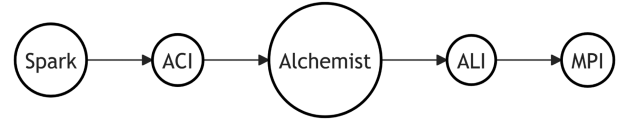


Fig. 7. The workflow of alchemist program execution.

ALI encapsulates and manages the MPI library. In practice, Spark tells Alchemist which MPI function it needs to use, and ALI is responsible for dynamically compiling the corresponding blocks into the program. This saves unnecessary compilation of MPI library functions and further improves efficiency. The execution process of the Alchemist program is shown in Fig. 7.

Programming Model: The Alchemist architecture needs to set the number of workers in the Alchemist cluster at startup. After the Alchemist cluster is started, the Driver node establishes a communication connection with the Driver node of the Spark cluster through the ACI interface, and exchanges cluster information through this connection. After that, the Worker nodes of the two clusters respectively establish connections according to the information of the Driver node, laying the foundation for subsequent data communication.

Algorithm 2 is an example to describe the use of Alchemist. First, it introduces the Alchemist and ALMatrix libraries, then creates the Alchemist computing environment and specifies the number of workers in the Alchemist cluster (line 1-2). Next, it registers the required MPI library for the newly created Alchemist computing environment. Here, it is assumed that the library containing the matrix multiplication algorithm is named ALlibAName and located at ALlibALocation (line 3). Then, it creates the calculated data based on the input data and starts the calculation. After the calculation is completed, it returns the result and terminates the Alchemist computing environment (line 4-7).

Performance: The implementation of ACI and ALI interfaces enables Alchemist to obtain very high performance. For matrix-related calculations, such as matrix multiplication and truncated singular value decomposition algorithm mentioned in the article [76], when the matrix scale reaches 50 GB, the calculation time of Spark has exceeded 30 min, while that of Alchemist has been kept within 7 min.

Compared to shared memory methods, socket communication avoids the overhead of replicating data across multiple nodes, reducing memory usage. However, it introduces its own challenges, such as reliance on network bandwidth and potential delays due to protocol overhead.

As the communication volume increases, network bandwidth and latency can become significant performance bottlenecks. To address these limitations, high-performance networking technologies such as RDMA can be introduced in subsequent designs, enabling direct memory access between nodes to minimize latency and maximize throughput [85], [86], [87].

Additionally, both shared memory and socket-based communication methods do not fully address the challenges of data serialization. For example, Alchemist primarily demonstrates serialization for floating-point matrices, leaving serialization for other data types unexplored.

B. Implementing Spark Mechanisms With MPI

Another integration approach to mitigate the overhead of data serialization is to extend MPI for implementing Big Data

Algorithm 2: Matrix-Vector Multiplications Using Alchemist.

Input: Matrix A , vector X
Output: Matrix B where $B = A \times X$

```

1: import alchemist.{Alchemist,
   AlMatrix}
2: ac ← new
   Alchemist.AlchemistContext(sc,
   numWorkers)
3: ac.registerLibrary(ALilibAName,
   ALilibALocation)
4: alA ← AlMatrix(A)
5: alX ← AlMatrix(X)
6: output ← ac.run(ALilibAName,
   multi_algs, alA, alX)
7: ac.stop()

```

processing mechanisms. Examples include FT-MRMPI [88], Thrill [77], MR-MPI [78], Mimir [89], [90], and DataMPI [79]. These frameworks leverage MPI's native communication capabilities to implement functionalities similar to Big Data frameworks while eliminating the need for explicit data serialization. This integration strategy provides tighter coupling between HPC and Big Data operations, improving performance in dual-intensive workloads.

1) *DataMPI*: DataMPI [79] is an architecture proposed by the Institute of Computing Technology, Chinese Academy of Sciences in 2014.

Architecture: The core idea of DataMPI architecture is to abstract the communication mechanism of Big Data framework into a 4D (Dichotomic, Dynamic, Data-centric, and Diversified) Bipartite Communication Model, and then use MPI communication mechanism to realize the Big Data mechanism, so as to improve the performance of Big Data processing.

The 4D Bipartite Communication Model divides the Big Data communication process into the Operation stage and the Aggregation stage. That is, communication occurs only between the Operation stage and the Aggregation stage, and the number of tasks in each stage changes dynamically, so no destination can be specified for communication. Moreover, The 4D Bipartite Communication Model also follows the principle of computing close to data.

The specific implementation idea of DataMPI is that DataMPI uses the **mpidrun** structure to call the process from the DataMPI pool and configure different environments to make the process run in communicator O or A as shown in Fig. 8. Each node can switch between O and A based on different configurations. That is, after a node executes a map task and stores data locally, it can switch to communicator A to continue to execute a reduce task. Data is read from the local node. Since all processes are uniformly managed by **mpidrun**, load balancing can be achieved through the scheduling algorithm and the task scheduling can follow the principle of computing close to the data, thereby reducing the performance loss caused by data communication between processes.

Programming Model: DataMPI provides the basic interfaces, as shown in Table IV. An example of implementing the Sort algorithm through DataMPI is shown in the algorithm 3.

The code starts by creating the DataMPI execution environment (line 1). Then, depending on the process type, the code in the map phase or the reduce phase is executed. The O

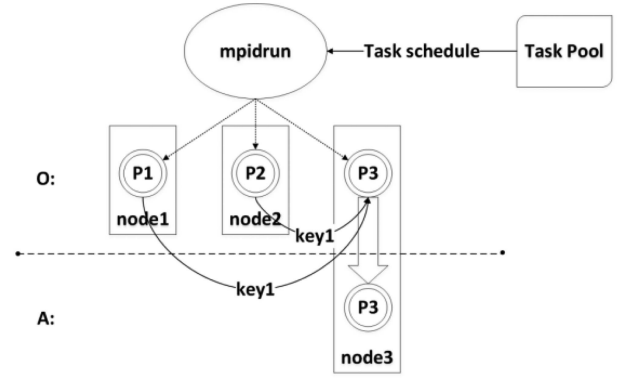


Fig. 8. Execution workflow of DataMPI.

TABLE IV
DATAMPI INTERFACE

Interface	Function
MPI_D_INIT	Initializing the environment
MPI_D_FINALIZE	The end of the environment
MPI_D_COMM_SIZE	Get the number of tasks
MPI_D_COMM_RANK	Gets the current task index
MPI_D_SEND	Send the KV
MPI_D_RECV	Receive KV
MPI_D_COMPARE	Comparison of the key
MPI_D_PARTITION	Partition KV
MPI_D_COMBINE	Combine KV

process immediately sends the result to the A process (line 2-10). The A process starts executing as soon as it receives data, without waiting for all the O processes to be finished (line 11-19). This has the benefit of maximizing the overlap between the map and reducing phases, as well as maximizing the overlap between computation and data transfer processes, enabling fine-grained concurrency and improving overall performance.

Performance: DataMPI uses MPI to implement the Big Data communication mechanism, and maximizes overlapping computing and communication through thread parallelism, thereby improving the overall performance. Experiments on Ethernet clusters with up to 65 nodes and five representative benchmarks (WordCount, TeraSort, PageRank, K-means, and Top-K) show that DataMPI improves job execution time by 31%-41% [79]. On the BigDataBench suite, DataMPI achieves up to 50% speedup compared to Spark [91]. In addition, DataMPI also provides good support for iterative and streaming computing. DataMPI-Iteration [92] further expands DataMPI and enhances its performance.

Compared with the integration mechanism of Spark calling MPI, DataMPI's integration idea of extending MPI avoids the data serialization overhead. However, DataMPI only optimizes the communication part. The management of Big Data and the fault tolerance mechanism are not mature enough and need further research.

C. Using a Cluster Resource Management System

Both Spark and MPI may undergo varying degrees of modification when using approaches like direct MPI invocation or extending MPI to implement Big Data mechanisms. These changes can disrupt the original architectures and introduce compatibility

Algorithm 3: Sorting Algorithm using DataMPI Architecture.

Input: Unordered array A
Output: Ordered array B

```

1: MPI_D.Init(args, MPI_D.Mode.Common,
   conf)
2: if MPI_D.COMM_BIPARTITE_O  $\neq$  null then
3:   rank  $\leftarrow$ 
     MPI_D.Comm_rank(MPI_D.COMM_BIPARTITE_O)
4:   size  $\leftarrow$ 
     MPI_D.Comm_size(MPI_D.COMM_BIPARTITE_O)
5:   keys  $\leftarrow$  loadKeys(rank, size)
6:   if keys  $\neq$  null then
7:     for  $i \leftarrow 1$  to keys.length do
8:       MPI_D.Send(keys[i], "")
9:     end for
10:  end if
11: else if MPI_D.COMM_BIPARTITE_A  $\neq$  null then
12:   rank  $\leftarrow$ 
     MPI_D.Comm_rank(MPI_D.COMM_BIPARTITE_A)
13:   size  $\leftarrow$ 
     MPI_D.Comm_size(MPI_D.COMM_BIPARTITE_A)
14:   keyValue  $\leftarrow$  MPI_D.Recv()
15:   while keyValue  $\neq$  null do
16:     outputKeyValue(rank, keyValue[0],
       keyValue[1])
17:     keyValue  $\leftarrow$  MPI_D.Recv()
18:   end while
19: end if
20: MPI_D.Finalize()

```

issues. To minimize the impact on the native ecosystems of Spark and MPI, an alternative integration strategy is to use a third-party cluster resource management system. This approach, exemplified by systems like myHadoop [80] and the Spark-MPI architecture [81], [93], facilitates communication between Spark and MPI clusters while requiring minimal alterations to the underlying frameworks. It allows for more flexible integration without significantly compromising the original functionality of either system.

1) *Spark-MPI*: Spark-MPI [81], [93] was jointly proposed by NSLS-II Department Brookhaven National Laboratory and Scientific Computing Kitware, Inc in 2017.

Architecture: The core design idea of spark-MPI architecture is to use the process startup and information exchange functions of cluster resource manager (such as SLURM [94] and HYDRA [95]) to establish MPI process communication between Spark workers, so as to realize the integration of Spark and MPI. The original Spark and MPI frameworks will not be changed.

The Fig. 9 shows the architecture of Spark-MPI. The worker node in the spark-MPI cluster is both a Spark worker and an MPI worker. After the user submits the code, Spark generates an RDD to partition the data, and then invokes the Spark interface to allocate the partitioned data to Worker nodes. After the Worker obtains the data and code, if it needs to run the MPI program, it obtains the information of other Worker nodes through PMI [96]. After all workers obtain all the information needed for MPI communication, they start the MPI process and run the MPI program through the *mpirun* standard command.

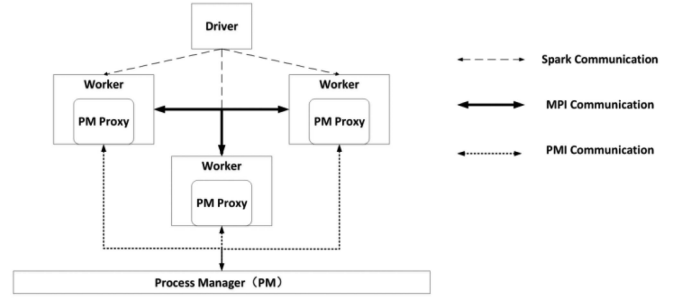


Fig. 9. Module design and interactions in Spark-MPI architecture.

Algorithm 4: Allreduce Algorithm Using Spark-MPI Architecture.

Input: Input data for reduction operation

Output: Aggregated result across all nodes

```

1: sc  $\leftarrow$  SparkContext()
2: envs  $\leftarrow$  read("pmixsrv.env") % Load global
   information of worker nodes via PMI
3: rdd  $\leftarrow$  sc.parallelize(envs,
   partitions)
4: comm  $\leftarrow$  MPI.COMM_WORLD % Switch to MPI for
   communication
5: comm.Allreduce(sendbuf, recvbuf, op
    $\leftarrow$  MPI.SUM)

```

Programming Model: Algorithm 4 outlines the implementation of the Allreduce operation using the Spark-MPI architecture. The process begins by configuring the PMI environment to provide global information about worker nodes (line 1-2). This information is then distributed across the cluster as an RDD using Spark (line 3). After initializing the MPI environment on the Spark Worker nodes, the MPI communication takes over to execute the Allreduce operation, aggregating data from all nodes efficiently (line 4-5).

Performance: Spark-MPI is simple to implement and framework-friendly, requiring minimal modifications to Spark and MPI. Additionally, on benchmarks for summing large arrays of floats across Spark workers, Spark-MPI/Infiniband can achieve performance improvements of up to two orders of magnitude compared to the Spark/Ethernet driver-worker model, depending on the number of nodes. It also demonstrates significant performance gains over the baseline method in applications such as Ptychographic and Tomographic imaging [97].

However, in a spark-MPI cluster, the workers of Spark and MPI are configured on the same node. Therefore, spark-MPI may have portability problems. In addition, this structure also faces the problem of data serialization.

D. Other Integration Methods

Beyond the integration approaches mentioned previously, there are other strategies, such as using MPI to directly replace communication primitives within Big Data frameworks to improve communication efficiency. This approach involves embedding MPI directly into the Big Data framework itself. While the result resembles the previously mentioned strategy of extending MPI to mimic Big Data framework operations, the underlying mechanisms differ.

TABLE V
SUMMARY OF INTEGRATED ARCHITECTURE

Name	Integration Degree	Comm. Load	Serialization	Scalability
Spark+MPI	low	high	yes	low
Alchemist	low	high	yes	medium
DataMPI	high	low	no	high
Spark-MPI	low	medium	yes	medium

Directly replacing communication primitives in Big Data frameworks with MPI presents several challenges:

- *Data structure standardization*: Big data frameworks and MPI rely on different data structures. Therefore, Spark's RDDs must be standardized or adapted to enable seamless MPI operation with Big Data framework data.
- *Global communication requirements*: MPI depends on global information for efficient inter-process communication, whereas Big Data frameworks abstract away process-level details, limiting direct access to such information.

MPIgnite [98] attempt to tackle these challenges by introducing SparkComm objects, which encapsulate the global information required for MPI communication. By leveraging Spark's `parallelizeFunc`, MPI code is distributed as function parameters to worker processes, allowing each parallel function to operate as an independent MPI instance.

Similarly, Chukonu [99] follows an alternative approach by implementing Spark's computing engine entirely in native languages while reusing Spark's mature ecosystem to avoid redundant development. MPI4Spark [100] replaces Spark's Netty-based communication mechanism with MPI, addressing Spark's shuffle-phase bottleneck and significantly improving performance in compute-intensive workloads. JAMPI [101] embeds MPI into Spark as algorithm-specific extensions, minimizing modifications to Spark's framework for simplicity at the cost of some flexibility.

While these architectures provide the necessary global information for MPI communication, they introduce scalability limitations and fail to fully resolve serialization challenges. Next steps could focus on enhancing scalability and improving compatibility between MPI and Big Data frameworks.

E. Summary

This section reviewed key integration techniques for combining Spark and MPI, examining several typical architectures. Each integration method offers unique advantages, but also has inherent limitations, as outlined in Table V.

When Spark invokes MPI, data transmission overhead is inevitable. Two key data transport modes—shared memory and socket communication—each have unique characteristics. Shared memory can result in multiple in-memory copies, leading to memory waste and latency due to replication. Therefore, the Spark+MPI architecture is better suited for dual-intensive applications that require multiple data communications. Socket-based communication, on the other hand, reduces memory duplication but can introduce network bottlenecks. Advances in high-speed networking (RDMA) may help mitigate these bottlenecks. However, serialization across data formats remains a challenge; for example, Alchemist supports floating-point matrix serialization, but broader format compatibility is needed.

Extending MPI to support Big Data mechanisms effectively addresses serialization issues, but this approach is still in its early stages. For example, DataMPI extends MPI's communication capabilities for Big Data workloads, yet it requires further development to evolve into a fully robust solution.

Each of these integration methods modifies the original Spark and MPI architectures to some extent, which can impact compatibility with the existing ecosystem. In the long term, these architectures may face challenges in achieving widespread adoption. Resource manager-based integration, while promising, is also in early development and requires improvements in fault tolerance and portability. Ultimately, to support universal usability and sustainability, integration architectures must continue to evolve into a comprehensive, user-friendly ecosystem.

VI. APPLICATION EXAMPLES

With advancements in integration architecture technology, these architectures are being applied across a range of fields. This section presents several typical application examples, highlighting how integrated architectures enhance performance and enable new capabilities in various domains.

A. Sequencing of Genome

Metagenomics aims to study the collective genomes of microbial populations within an environment, often requiring genomic sequencing of highly heterogeneous communities. These data sets can encompass over 10,000 microbial species, often containing noise and incomplete sequences, making analysis complex. From sampling and genome assembly to gene and functional predictions, metagenomics presents significant computational and data-processing challenges due to its extensive data volume and the need for high-precision analytics, classifying it as a dual-intensive field [102].

To address these challenges, DIME [103] introduced a metagenomic sequence assembly framework employing a divide-and-conquer approach. However, it relies solely on the Hadoop architecture, without leveraging hybrid architectures. In contrast, Han Lin et al. [104] utilized a Direct MPI Invocation in Spark architecture, where data transfer is managed via HDFS. This approach achieved an average acceleration of 9.6 times compared to the initial implementation, aided by additional optimization measures. This integration of Spark and MPI enables high-throughput data handling alongside efficient computational capabilities, proving especially suitable for complex, large-scale metagenomic analyses.

B. Precision Agriculture

Precision agriculture (PA) optimizes agricultural production by leveraging data on crop growth factors such as wind, temperature, humidity, fertilizer supply, and soil conditions to minimize waste and costs [105]. The diverse data collection methods, various data types, and large data volumes make predicting crop growth a dual-intensive task involving both data and computation.

Faced with these challenges, the CYBELE platform [106] integrates cloud environments (supporting Big Data frameworks such as Spark) with HPC clusters (MPI) using a hybrid Cluster Resource Management System. This architecture combines Kubernetes [107] with HPC resource managers like

TORQUE [108] and SLURM [94] using plugins such as Torque-Operator and WLM-Operator, enabling efficient resource allocation and job scheduling. The platform has demonstrated its effectiveness in PA through various evaluations. Perakis et al. [109] conducted nine experiments across different agricultural and animal husbandry domains, highlighting the platform's ability to advance PA applications. Georgiou et al. [110] further showcased a PA workflow using CYBELE, exemplified by a wheat ear counting application in the field. Their experiments, compared to the original implementation, demonstrated a 100-fold performance improvement, highlighting the platform's ability to accelerate dual-intensive agricultural tasks.

C. Disaster Warning

Floods are among the most devastating natural disasters, causing significant economic losses and casualties worldwide. For instance, the "7·20" Extreme Rainfall and Flooding Event in Zhengzhou led to hundreds of deaths [111]. Effective monitoring and accurate prediction are crucial to mitigating such impacts. Flood prediction applications rely on the integration of large-scale, multi-dimensional hydrological data, which exhibit all-weather, continuous, and multifactorial characteristics, demanding both efficient data handling and substantial computational power [112], [113].

Integrated computing frameworks that merge cluster and cloud computing with HPC and Big Data technologies, such as LEXIS [114], are crucial for flood monitoring, enabling real-time data processing, predictive modeling, and rapid decision-making to enhance disaster response [115]. Similarly, Liu et al. [116] propose an HDA+HPC geocomputation architecture that integrates high-end data analytics (HDA) for spatial data management with HPC for numerical modeling, leveraging solutions like Alchemist, Spark+MPI, and Spark-MPI, which were analyzed earlier. This approach effectively addresses scalability challenges in flood inundation mapping involving large-scale geospatial datasets.

VII. CHALLENGES AND OPEN ISSUES

This section highlights key research challenges and outlines potential opportunities in integration technologies for Big Data and high-performance computing. Addressing these challenges is crucial for advancing the development and deployment of integrated architectures.

Application-Specific Adaptability: Current integration frameworks often require significant adjustments for each application, particularly in handling diverse data serialization formats and task execution models. For example, the lack of standardization across frameworks complicates resource scheduling, data exchange, and fault recovery processes. To overcome these challenges, future research should focus on developing a universal, structure-centered ecosystem. Such an ecosystem would include unified resource schedulers, standardized file systems, and consistent development interfaces, fostering improved usability, interoperability, and scalability across diverse applications.

Utilization of Heterogeneous Architectures: With the proliferation of CPU+GPU hybrid clusters, integration frameworks face increasing pressure to fully leverage GPU capabilities for compute-intensive tasks. While existing frameworks, such as SWAT [117], and hybrid approaches [118], [119] leveraging OpenCL [120] or OpenACC [121], have made strides in

this area, they often suffer from shallow integration. These methods frequently result in performance bottlenecks, particularly during data transfer between the Big Data framework and GPUs. Future work should prioritize deeper integration that aligns GPU memory management and task scheduling with Big Data processing workflows. Techniques such as RDMA-enabled GPU communication and optimized memory pipelines could play a critical role in eliminating these bottlenecks.

Improving Load Balancing Across Nodes: Traditional Big Data frameworks emphasize data locality to reduce data movement, but this can inadvertently create imbalanced workloads across computing nodes. For integrated architectures, achieving efficient load balancing is essential to fully utilize resources. Future research should investigate scheduling strategies that dynamically adapt to workload characteristics, balancing computational and I/O demands across nodes. For instance, hybrid scheduling algorithms that combine data locality awareness with predictive load-balancing techniques could significantly enhance system performance.

Unified Fault-Tolerance Framework: In distributed Big Data processing, fault tolerance is typically ensured through data redundancy and replication, whereas long-running HPC tasks often rely on checkpointing to periodically save computational states for recovery [122]. Integrating Big Data frameworks with HPC technologies poses a challenge in reconciling these different resilience strategies. A hybrid approach that combines redundancy and checkpointing may offer greater flexibility, while a more advanced solution could involve intelligent runtime adaptation, dynamically selecting the optimal strategy based on workload characteristics. Future research should focus on developing a unified and efficient recovery framework that leverages multiple fault-tolerance mechanisms.

As emerging fields like artificial intelligence, genomics, and climate modeling increasingly require dual-intensive (data- and compute-intensive) processing, integrated architectures are poised for broader adoption. Advances in hardware technologies, including faster interconnects, non-volatile memory, and specialized accelerators, will likely drive innovation in integration frameworks. Research efforts should also focus on leveraging these advancements to achieve seamless orchestration between Big Data frameworks and HPC systems, unlocking new capabilities and addressing scalability challenges.

VIII. DISCUSSION ON OTHER INTEGRATIONS

While research on integration techniques has predominantly centered around combining Spark with MPI, there is growing interest in exploring alternative integrations, such as merging other Big Data frameworks (e.g., Flink [123], Storm [124]) with HPC technologies (e.g., OpenMP [125]). These efforts aim to harness the strengths of both domains, sharing methodological parallels yet introducing unique characteristics. Such approaches hold significant research value in specific application scenarios, offering opportunities for cross-referencing.

Flink and Spark, as memory-based distributed frameworks, share many similarities, enabling transferable design and optimization techniques. Chen et al. [126] introduced GFLink, an extension of Flink integrating heterogeneous CPU-GPU clusters, substantially boosting its computational capabilities. Building on this, a parallel Hierarchical Extreme Learning Machine for processing large-scale datasets with GPU acceleration was

introduced [127]. To address the limitations of GPU-specific implementations like CUDA, FlinkCL [118] was developed, leveraging OpenCL to expand hardware compatibility and improve scalability.

Stream processing frameworks are increasingly optimized through GPU acceleration, RDMA-based data transmission, and MPI-based distributed computing. Rathore et al. [128] enhanced real-time Big Data stream processing by integrating Spark with GPUs in a Hadoop-based distributed environment, significantly outperforming traditional CPU-based MapReduce. Zhang et al. [129] optimized Apache Storm's communication module with RDMA, reducing CPU load and boosting throughput. Meanwhile, frameworks like AIR [130] and a hybrid MPI-based stream processing system [131] highlight the effectiveness of MPI and other HPC techniques in improving stream processing performance.

OpenMP, traditionally used in shared-memory HPC environments, has also been integrated into Big Data contexts. OmpCloud [132], [133] combines OpenMP directives with Spark-based parallelization and remote communication management. By extending OpenMP's accelerator model, OmpCloud allows clusters to be treated as computing devices, simplifying the development of distributed parallel programs. For dynamic programming problems like the Gaussian Elimination Paradigm, Javanmard et al. [134] proposed Spark implementations that offload computations to OpenMP environments, demonstrating seamless integration and notable performance gains.

These advancements illustrate the potential of combining Big Data frameworks with HPC technologies, paving the way for future innovations in distributed and heterogeneous computing environments.

IX. CONCLUSION

As workloads increasingly exhibit both data-intensive and compute-intensive characteristics, the integration of Big Data frameworks with high-performance computing has become a critical and evolving research area. This integration aims to create computing architectures that excel in multiple dimensions, including computing power, storage performance, network transmission, data and computational resilience, and scalability. While the integration of Big Data frameworks such as Spark with HPC technologies like MPI has gained significant attention, it faces numerous challenges related to data transmission, serialization, complexity, compatibility, and flexibility. This survey categorizes the main integration approaches into three types: Direct MPI invocation in Spark, Implementing Spark mechanisms with MPI, and Using a cluster resource management system. Each approach presents its own strengths and trade-offs. Direct MPI invocation in Spark is easy to implement but suffers from issues like inter-cluster communication and data serialization. Implementing Spark mechanisms with MPI offers high performance but is complex and faces compatibility issues with standard frameworks. Using a cluster resource management system introduces minimal impact on the native frameworks but adds new dependencies, which can limit flexibility. Despite these challenges, the discussed integration solutions have demonstrated considerable practical value and have clear paths for further optimization. This survey aims to serve as a valuable resource for researchers and practitioners, advancing the development of more efficient, scalable, and robust computing systems.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their valuable feedback, which greatly enhanced the quality of our paper. Some grammatical errors are corrected using ChatGPT.

REFERENCES

- [1] E. Korkmaz, "Adversarial robust deep reinforcement learning requires redefining robustness," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 8369–8377.
- [2] A. Chen, N. Gisolfi, and A. Dubrawski, "Data-driven discovery of design specifications (student abstract)," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 23449–23450.
- [3] S. Goyal, P. Maini, Z. C. Lipton, A. Raghunathan, and J. Z. Kolter, "Scaling laws for data filtering—data curation cannot be compute agnostic," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 22702–22711.
- [4] L. Chen, F. Du, Y. Hu, Z. Wang, and F. Wang, "Swinrdm: Integrate swinrm with diffusion model towards high-resolution and high-quality weather forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 322–330.
- [5] T. Nguyen et al., "Scaling transformer neural networks for skillful and reliable medium-range weather forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang Eds., 2024, vol. 37, pp. 68740–68771.
- [6] O. U. Onwuka and A. Adu, "Technological synergies for sustainable resource discovery: Enhancing energy exploration with carbon management," *Eng. Sci. Technol. J.*, vol. 5, no. 4, pp. 1203–1213, 2024.
- [7] S. Zhao et al., "The role of satellite remote sensing in mitigating and adapting to global climate change," *Sci. Total Environ.*, vol. 904, 2023, Art. no. 166820.
- [8] M. Alibeigi et al., "Zenseact open dataset: A large-scale and diverse multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 20178–20188.
- [9] J. Yuan et al., "Ad-pt: Autonomous driving pre-training with large-scale point cloud dataset," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2023, pp. 47914–47933.
- [10] E. Nguyen et al., "HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2023, pp. 43177–43201.
- [11] C. C. Wanjura and F. Marquardt, "Fully nonlinear neuromorphic computing with linear wave scattering," *Nature Phys.*, vol. 20, pp. 1434–1440, 2024.
- [12] M. Torabzadehkashi, S. Rezaei, A. Heydari-Gorji, H. Bobarshad, V. Alves, and N. Bagherzadeh, "Computational storage: An efficient and scalable platform for Big Data and hpc applications," *J. Big Data*, vol. 6, no. 1, 2019, Art. no. 100.
- [13] S. Jha et al., "Live forensics for HPC systems: A case study on distributed storage systems," in *Proc. Int. Conf. High Perform. Computing, Netw., Storage Anal.*, 2020, pp. 1–16.
- [14] N. Dryden, R. Böhringer, T. Ben-Nun, and T. Hoefler, "Clairvoyant prefetching for distributed machine learning I/O," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, New York, NY, USA, 2021, pp. 1–15.
- [15] M. Zhao et al., "Understanding data storage and ingestion for large-scale deep recommendation model training: Industrial product," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, New York, NY, USA, 2022, pp. 1042–1057.
- [16] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Comput. Surv.*, vol. 55, no. 6, pp. 1–29, Dec. 2022.
- [17] A. K. Sandhu, "Big data with cloud computing: Discussions and challenges," *Big Data Mining Analytics*, vol. 5, no. 1, pp. 32–40, 2022.
- [18] L. Helali and M. N. Omri, "A survey of data center consolidation in cloud computing systems," *Comput. Sci. Rev.*, vol. 39, 2021, Art. no. 100366.
- [19] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop Hot Topics Cloud Comput.*, Boston, MA, USA, 2010, p. 10.
- [20] W. Gropp, W. D. Gropp, E. Lusk, A. Skjellum, and A. D. F. E. Lusk, *Using MPI: Portable Parallel Programming With the Message-Passing Interface*, vol. 1. Cambridge, MA, USA: MIT Press, 1999.

- [21] A. M. Iktun, A. E. Ezugwu, L. Abualigah, B. Abuhaija, and J. Heming, "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the ERA of Big Data," *Inf. Sci.*, vol. 622, pp. 178–210, 2023.
- [22] G. M. Slota, S. Rajamanickam, and K. Madduri, "A case study of complex graph analysis in distributed memory: Implementation and optimization," in *Proc. 2016 IEEE Int. Parallel Distrib. Process. Symp.*, 2016, pp. 293–302.
- [23] A. Gittens et al., "Matrix factorizations at scale: A comparison of scientific data analytics in spark and C MPI using three case studies," in *Proc. 2016 IEEE Int. Conf. Big Data*, 2016, pp. 204–213.
- [24] A. Gittens et al., "Matrix factorizations at scale: A comparison of scientific data analytics in spark and C MPI using three case studies," in *Proc. 2016 IEEE Int. Conf. Big Data*, 2016, pp. 204–213.
- [25] J. L. Reyes-Ortiz, L. Oneto, and D. Anguita, "Big Data analytics in the cloud: Spark on hadoop vs MPI/OpenMP on Beowulf," *Procedia Comput. Sci.*, vol. 53, pp. 121–130, 2015.
- [26] M. Saxena, S. Jha, S. Khan, J. Rodgers, P. Lindner, and E. Gabriel, "Comparison of MPI and spark for data science applications," in *Proc. 2020 IEEE Int. Parallel Distrib. Process. Symp. Workshops*, 2020, pp. 682–690.
- [27] J. Qiu, S. Jha, A. Luckow, and G. C. Fox, "Towards hpc-abds: An initial high-performance Big Data stack," *Buil. Robust Big Data Ecosystem ISO/IEC JTC*, vol. 1, pp. 18–21, 2014.
- [28] S. Mei, H. Guan, and Q. Wang, "An overview on the convergence of high performance computing and Big Data processing," in *Proc. IEEE 24th Int. Conf. Parallel Distrib. Syst.*, 2018, pp. 1046–1051.
- [29] S. Usman, R. Mehmood, and I. Katib, "Big data and HPC convergence for smart infrastructures: A review and proposed architecture," in *Proc. Smart Infrastructure Appl.*, 2020, pp. 561–586.
- [30] V.-H. Le and H. Zhang, "Log-based anomaly detection with deep learning: How far are we?," in *Proc. 44th Int. Conf. Softw. Eng.*, New York, NY, USA, 2022, pp. 1356–1367.
- [31] E. Mehmood and T. Anees, "Distributed real-time ETL architecture for unstructured Big Data," *Knowl. Inf. Syst.*, vol. 64, no. 12, pp. 3419–3445, 2022.
- [32] Y. Li, X. Lyu, N. Koren, L. Lyu, B. Li, and X. Ma, "Anti-backdoor learning: Training clean models on poisoned data," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2021, pp. 14900–14912.
- [33] A. Jackowski et al., "ObjDedup: High-throughput object storage layer for backup systems with block-level deduplication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 7, pp. 2180–2197, Jul. 2023.
- [34] N. Nivedhaa, "Towards efficient data migration in cloud computing: A comparative analysis of methods and tools," *Int. J. Artif. Intell. Cloud Comput.*, vol. 2, no. 1, pp. 1–16, 2024.
- [35] R. Sahal, J. G. Breslin, and M. I. Ali, "Big data and stream processing platforms for industry 4.0 requirements mapping for a predictive maintenance use case," *J. Manuf. Syst.*, vol. 54, pp. 138–151, 2020.
- [36] Y. Deldjoo, M. Schedl, P. Cremonesi, and G. Pasi, "Recommender systems leveraging multimedia content," *ACM Comput. Surv.*, vol. 53, no. 5, pp. 1–38, Sep. 2020.
- [37] Y. Han et al., "A short-term wind speed prediction method utilizing novel hybrid deep learning algorithms to correct numerical weather forecasting," *Appl. Energy*, vol. 312, 2022, Art. no. 118777.
- [38] S. Partee et al., "Using machine learning at scale in numerical simulations with smartsim: An application to ocean climate modeling," *J. Comput. Sci.*, vol. 62, 2022, Art. no. 101707.
- [39] O. Adjoua et al., "Tinker-HP: Accelerating molecular dynamics simulations of large complex systems with advanced point dipole polarizable force fields using GPUs and multi-GPU systems," *J. Chem. Theory Comput.*, vol. 17, no. 4, pp. 2034–2053, 2021, PMID: 33755446.
- [40] J. Fang et al., "Direct numerical simulation of reactor two-phase flows enabled by high-performance computing," *Nucl. Eng. Des.*, vol. 330, pp. 409–419, 2018.
- [41] J. Domke et al., "Matrix engines for high performance computing: A paragon of performance or grasping at straws?," in *Proc. 2021 IEEE Int. Parallel Distrib. Process. Symp.*, 2021, pp. 1056–1065.
- [42] K. Jurczuk, M. Czajkowski, and M. Kretowski, "Multi-GPU approach to global induction of classification trees for large-scale data mining," *Appl. Intell.*, vol. 51, no. 8, pp. 5683–5700, 2021.
- [43] A. Bojchevski et al., "Scaling graph neural networks with approximate pagerank," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2020, pp. 2464–2473.
- [44] X. Zhao, J. Liang, and J. Wang, "A community detection algorithm based on graph compression for large-scale social networks," *Inf. Sci.*, vol. 551, pp. 358–372, 2021.
- [45] K. Osawa, Y. Tsuji, Y. Ueno, A. Naruse, C.-S. Foo, and R. Yokota, "Scalable and practical natural gradient for large-scale deep learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 404–415, Jan. 2022.
- [46] L. Wang et al., "Diesel: A dataset-based distributed storage and caching system for large-scale deep learning training," in *Proc. 49th Int. Conf. Parallel Process.*, New York, NY, USA, 2020, pp. 1–11.
- [47] J. Miraclin Joyce Pamila, R. SenthilP. SelviSanthi, and T. Nithya, "Ensemble classifier based Big Data classification with hybrid optimal feature selection," in *Proc. Adv. Eng. Softw.*, 2022, Art. no. 103183.
- [48] D. Huang, F. Jiang, K. Li, G. Tong, and G. Zhou, "Scaled PCA: A new approach to dimension reduction," *Manage. Sci.*, vol. 68, no. 3, pp. 1678–1695, 2022.
- [49] R. C. Edgar et al., "Petabase-scale sequence alignment catalyses viral discovery," *Nature*, vol. 602, no. 7895, pp. 142–147, 2022.
- [50] C. K. Leung, O. A. Sarumi, and C. Y. Zhang, "Predictive analytics on genomic data with high-performance computing," in *Proc. IEEE Int. Conf. Bioinf. Biomed.*, 2020, pp. 2187–2194.
- [51] B. Corcoran et al., "Ultra-dense optical data transmission over standard fibre with a single chip source," *Nature Commun.*, vol. 11, no. 1, 2020, Art. no. 2568.
- [52] G. F. Pfister, "An introduction to the infiniband architecture," *High Perform. Mass Storage Parallel I/O*, vol. 42, no. 617–632, 2001, Art. no. 102.
- [53] Y. Gao et al., "When cloud storage meets RDMA," in *Proc. 18th USENIX Symp. Netw. Syst. Des. Implementation*, 2021, pp. 519–533.
- [54] J. Dean and S. Ghemawat, "Mapreduce: Simplified data processing on large clusters," *Commun. ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [55] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Syst. Technol.*, 2010, pp. 1–10.
- [56] R. L. Graham, T. S. Woodall, and J. M. Squyres, "Open MPI: A flexible high performance MPI," in *Proc. Int. Conf. Parallel Process. Appl. Math.*, 2005, pp. 228–239.
- [57] W. Gropp and E. Lusk, "User's guide for mpich, a portable implementation of MPI," 1996, pp. 1–39.
- [58] K. Bakshi, "Considerations for Big Data: Architecture and approach," in *Proc. Aerosp. Conf.*, 2012, pp. 1–7.
- [59] V. Eijkhout, *Introduction to High Performance Scientific Computing*. Durham, NC, USA: Lulu.com, 2010.
- [60] F. Yin and F. Shi, "A comparative survey of Big Data computing and HPC: From a parallel programming model to a cluster architecture," *Int. J. Parallel Program.*, vol. 50, no. pp. 27–64, 2022.
- [61] H. Asaadi, D. Khaldi, and B. Chapman, "A comparative survey of the HPC and Big Data paradigms: Analysis and experiments," in *Proc. 2016 IEEE Int. Conf. Cluster Comput.*, 2016, pp. 423–432.
- [62] S. Elmisaoui, I. Kissami, and J.-M. Ghidaglia, "High-performance computing to accelerate large-scale computational fluid dynamics simulations: A comprehensive study," in *Proc. Int. Conf. Adv. Intell. Syst. Sustain. Develop.*, 2023, pp. 352–360.
- [63] S. P. Zwart, "The ecological impact of high-performance computing in astrophysics," *Nature Astron.*, vol. 4, no. 9, pp. 819–822, Sep. 2020.
- [64] J. Gray, D. T. Liu, M. Nieto-Santesteban, A. Szalay, D. J. DeWitt, and G. Heber, "Scientific data management in the coming decade," *Acm Sigmod Rec.*, vol. 34, no. 4, pp. 34–41, 2005.
- [65] C. A. Lee, S. D. Gasster, A. Plaza, C. I. Chang, and B. Huang, "Recent developments in high performance computing for remote sensing: A review," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 4, no. 3, pp. 508–527, Sep. 2011.
- [66] N. Ritter and M. Ruth, "The geotiff data interchange standard for raster geographic images," *Int. J. Remote Sens.*, vol. 18, no. 7, pp. 1637–1647, 1997.
- [67] S. Koranne, *Hierarchical Data Format 5 : Hdf5*. Berlin, Germany: Springer, 2011, pp. 191–200.
- [68] R. Rew and G. Davis, "NetCDF: An interface for scientific data access," *IEEE Comput. Graph. Appl.*, vol. 10, no. 4, pp. 76–82, Jul. 1990.
- [69] J. Liu, E. Racah, Q. Koziol, R. S. Canon, and M. Prabhat, "H5spark: Bridging the I/O gap between spark and scientific data formats on HPC systems," *Cray User Group*, 2016, pp. 1–8.
- [70] P. Xuan, J. Denton, P.K. Srimani, R. Ge, and F. Luo, "Big data analytics on traditional HPC infrastructure using two-level storage," in *Proc. 2015 Int. Workshop Data-Intensive Scalable Comput. Syst.*, 2015, pp. 1–8.
- [71] Q. Chen, K. Chen, Z.-N. Chen, W. Xue, X. Ji, and B. Yang, "Lessons learned from optimizing the sunway storage system for higher application I/O performance," *J. Comput. Sci. Technol.*, vol. 35, pp. 47–60, 2020.
- [72] A. K. Paul et al., "I/O load balancing for Big Data hpc applications," in *Proc. 2017 IEEE Int. Conf. Big Data*, 2017, pp. 233–242.

- [73] N. S. Islam, D. Shankar, X. Lu, M. Wasi-Ur-Rahman, and D. K. Panda, "Accelerating I/O performance of Big Data analytics on HPC clusters through RDMA-based key-value store," in *Proc. IEEE 44th Int. Conf. Parallel Process.*, 2015, pp. 280–289.
- [74] P. Xuan, W. B. Ligon, P.K. Srimani, R. Ge, and F. Luo, "Accelerating Big Data analytics on HPC clusters using two-level storage," *Parallel Comput.*, vol. 61, pp. 18–34, 2017.
- [75] M. Anderson et al., "Bridging the gap between HPC and Big Data frameworks," *Proc. VLDB Endowment*, vol. 10, no. 8, pp. 901–912, 2017.
- [76] A. Gittens et al., "Alchemist: An apache spark mpi interface," *Concurrency Comput.: Pract. Experience*, vol. 31, no. 16, 2019, Art. no. e5026.
- [77] T. Bingmann et al., "Thrill: High-performance algorithmic distributed batch data processing with C," in *Proc. 2016 IEEE Int. Conf. Big Data*, 2016, pp. 172–183.
- [78] S. J. Plimpton and K. D. Devine, "Mapreduce in MPI for large-scale graph algorithms," *Parallel Comput.*, vol. 37, no. 9, pp. 610–632, 2011.
- [79] X. Lu, F. Liang, B. Wang, L. Zha, and Z. Xu, "Datampi: Extending MPI to hadoop-like Big Data computing," in *Proc. IEEE 28th Int. Parallel Distrib. Process. Symp.*, 2014, pp. 829–838.
- [80] S. Krishnan, M. Tatineni, and C. Baru, "myhadoop-hadoop-on-demand on traditional HPC resources," San Diego Supercomputer Center, Tech. Rep. TR-2011-2, University of California, San Diego, 2011, Art. no. 9.
- [81] N. Malitsky, "Bringing the HPC reconstruction algorithms to Big Data platforms," in *Proc. 2016 New York Sci. Data Summit*, 2016, pp. 1–8.
- [82] N. Mammadli, J. Ejorque, J. Alvarez, and R. M. Badia, "DDS: Integrating data analytics transformations in task-based workflows," *Open Res. Europe*, vol. 2, 2023, Art. no. 66.
- [83] H. Li, "Alluxio: A virtual distributed file system," University of California, Berkeley, 2018.
- [84] A. Gittens et al., "Accelerating large-scale data analysis by offloading to high-performance computing libraries using alchemist," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 293–301.
- [85] A. Woodie, "Does infiniband have a future on hadoop," *Datanami*, vol. 4, 2015.
- [86] X. Lu, M. W. U. Rahman, N. Islam, D. Shankar, and D. K. Panda, "Accelerating spark with rdma for Big Data processing: Early experiences," in *Proc. IEEE 22nd Annu. Symp. High- Perform. Interconnects*, 2014, pp. 9–16.
- [87] Y. Wang et al., "Assessing the performance impact of high-speed interconnects on mapreduce," in *Specifying Big Data Benchmarks*. Berlin, Germany: Springer, 2012, pp. 148–163.
- [88] Y. Guo, W. Bland, P. Balaji, and X. Zhou, "Fault tolerant mapreduce-MPI for HPC clusters," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, New York, NY, USA, 2015, pp. 1–12.
- [89] T. Gao et al., "Mimir: Memory-efficient and scalable mapreduce for large supercomputing systems," in *Proc. 2017 IEEE Int. Parallel Distrib. Process. Symp.*, 2017, pp. 1098–1108.
- [90] T. Gao et al., "Memory-efficient and skew-tolerant mapreduce over MPI for supercomputing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 12, pp. 2734–2748, Dec. 2020.
- [91] F. Liang, C. Feng, X. Lu, and Z. Xu, "Performance benefits of datampi: A case study with bigdatabench," in *Proc. Workshop Big Data Benchmarks, Perform. Optimization, Emerg. Hardware*, 2014, pp. 111–123.
- [92] F. Liang and X. Lu, "Accelerating iterative Big Data computing through MPI," *J. Comput. Sci. Technol.*, vol. 30, no. 2, pp. 283–294, 2015.
- [93] N. Malitsky, R. Castain, and M. Cowan, "Spark-MPI: Approaching the fifth paradigm of cognitive applications," 2018, *arXiv: 1806.01110*.
- [94] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple linux utility for resource management," in *Proc. Workshop Job Scheduling Strategies Parallel Process.*, 2003, pp. 44–60.
- [95] C. Curino et al., "Hydra: A federated resource manager for data-center scale analytics," in *Proc. 16th USENIX Symp. Netw. Syst. Des. Implementation*, 2019, pp. 177–192.
- [96] P. Balaji et al., "PMI: A scalable parallel process-management interface for extreme-scale systems," in *Proc. Eur. MPI Users Group Meeting*, 2010, pp. 31–41.
- [97] N. Malitsky et al., "Building near-real-time processing pipelines with the spark-MPI platform," in *Proc. 2017 New York Sci. Data Summit*, 2017, pp. 1–8.
- [98] B. L. Morris and A. Skjellum, "Mpignite: An MPI-like language and prototype implementation for apache spark," 2017, *arXiv: 1707.04788*.
- [99] B. Yu et al., "Chukonu: A fully-featured high-performance Big Data framework that integrates a native compute engine into spark," *Proc. VLDB Endow.*, vol. 15, no. 4, pp. 872–885, Dec. 2021.
- [100] K. Al-Attar, A. Shafi, M. Abduljabbar, H. Subramoni, and D. K. Panda, "Spark meets MPI: Towards high-performance communication framework for spark using MPI," in *Proc. 2022 IEEE Int. Conf. Cluster Comput.*, 2022, pp. 71–81.
- [101] T. Foldi, C. von Csefalvay, and N. A. Perez, "JAMPI: Efficient matrix multiplication in spark using barrier execution mode," *Big Data Cogn. Comput.*, vol. 4, no. 4, 2020, Art. no. 32.
- [102] J. C. Wooley, A. Godzik, and I. Friedberg, "A primer on metagenomics," *PLoS Comput. Biol.*, vol. 6, no. 2, 2010, Art. no. e1000667.
- [103] X. Guo, N. Yu, X. Ding, J. Wang, and Y. Pan, "Dime: A novel framework for de novo metagenomic sequence assembly," *J. Comput. Biol.: A J. Comput. Mol. Cell Biol.*, vol. 222, pp. 159–77, 2015.
- [104] H. Lin et al., "Combining hadoop with MPI to solve metagenomics problems that are both data-and compute-intensive," *Int. J. Parallel Program.*, vol. 46, no. 4, pp. 762–775, 2018.
- [105] F. J. Pierce and P. Nowak, "Aspects of precision agriculture," *Adv. Agronomy*, vol. 67, pp. 1–85, 1999.
- [106] N. Zhou et al., "Cybele: A Hybrid Architecture of Hpc and Big Data for Ai Applications in Agriculture," in *HPC, Big Data, and AI Convergence Towards Exascale*. Boca Raton, FL, USA: CRC Press, 2022, pp. 255–272.
- [107] The Kubernetes Authors, "Kubernetes: Production-grade container orchestration," 2024. [Online]. Available: <https://kubernetes.io>
- [108] Adaptive Computing Enterprises, Inc., "TORQUE resource manager," 2009. [Online]. Available: <https://adaptivecomputing.com/cherry-services/torque-resource-manager/>
- [109] K. Perakis, F. Lampathaki, K. Nikas, Y. Georgiou, O. Marko, and J. Maselyne, "Cybele-fostering precision agriculture & livestock farming through secure access to large-scale HPC enabled virtual industrial experimentation environments fostering scalable Big Data analytics," *Comput. Netw.*, vol. 168, 2020, Art. no. 107035.
- [110] Y. Georgiou et al., "Converging HPC, Big Data and cloud technologies for precision agriculture data analytics on supercomputers," in *Proc. Int. Conf. High Perform. Comput.*, Springer, 2020, pp. 368–379.
- [111] Z. Chen, F. Kong, and M. Zhang, "A case study of the '7–20,' extreme rainfall and flooding event in zhengzhou, henan province, china from the perspective of fragmentation," *Water*, vol. 14, no. 19, 2022, Art. no. 2970.
- [112] X. Song et al., "Big data and emergency management: Concepts, methodologies, and applications," *IEEE Trans. Big Data*, vol. 8, no. 2, pp. 397–419, Apr. 2022.
- [113] S. K. Sood, R. Sandhu, K. Singla, and V. Chang, "IoT, Big Data and HPC based smart flood management framework," *Sustain. Comput.: Inform. Syst.*, vol. 20, pp. 102–117, 2018.
- [114] A. Scionti et al., "HPC, cloud and big-data convergent architectures: The lexis approach," in *Complex, Intelligent, and Software Intensive Systems*, L. Barolli, F. K. Hussain, and M. Ikeda Eds. Berlin, Germany: Springer, 2020, pp. 200–212.
- [115] V. M. Quiroga, I. Popescu, D. P. Solomatine, and L. Bociort, "Cloud and cluster computing in uncertainty analysis of integrated flood models," *J. Hydroinformatics*, vol. 15, no. 1, pp. 55–70, 2012.
- [116] Y. Y. Liu and J. Sanyal, "Scalable data-intensive geocomputation: A design for real-time continental flood inundation mapping," in *Driving Scientific and Engineering Discoveries Through the Convergence of HPC, Big Data and AI*. Berlin, Germany: Springer, 2020, pp. 130–144.
- [117] M. Grossman and V. Sarkar, "Swat: A programmable, in-memory, distributed, high-performance computing platform," in *Proc. 25th ACM Int. Symp. High- Perform. Parallel Distrib. Comput.*, 2016, pp. 81–92.
- [118] C. Chen, K. Li, A. Ouyang, and K. Li, "FlinkCL: An openCL-based in-memory computing architecture on heterogeneous CPU-GPU clusters for Big Data," *IEEE Trans. Comput.*, vol. 67, no. 12, pp. 1765–1779, Dec. 2018.
- [119] S. Alshahrani, W. AlJ. Shehri, A. M. Almaliki, Alghamdi, and A. M. Alam-mari, "Accelerating spark-based applications with MPI and openacc," *Complexity*, vol. 2021, 2021, Art. no. 9943289.
- [120] A. Munshi, B. Gaster, T. G. Mattson, and D. Ginsburg, *OpenCL Programming Guide*. Tamil Nadu, India: Pearson Education, 2011.
- [121] R. Farber, *Parallel Programming With OpenACC*. Boston, USA: Newnes, 2016.
- [122] S. Isukapalli and S.N. Srirama, "A systematic survey on fault-tolerant solutions for distributed data analytics: Taxonomy, comparison, and future directions," *Comput. Sci. Rev.*, vol. 53, 2024, Art. no. 100660.
- [123] Apache Flink, "Apache flink documentation," 2024, Accessed: Nov. 28, 2024. [Online]. Available: <https://flink.apache.org/documentation.html>
- [124] R. Evans, "Apache storm, a hands on tutorial," in *Proc. 2015 IEEE Int. Conf. Cloud Eng.*, 2015, pp. 2–2.

- [125] O. A. R. Board, "Openmp application programming interface (API) version 5.2," 2020, Accessed: Nov. 28, 2024. [Online]. Available: <https://www.openmp.org/specifications/>
- [126] C. Chen, K. Li, A. Ouyang, Z. Zeng, and K. Li, "GFLink: An in-memory computing architecture on heterogeneous CPU-GPU clusters for Big Data," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 6, pp. 1275–1288, Jun. 2018.
- [127] C. Chen, K. Li, A. Ouyang, Z. Tang, and K. Li, "GPU-accelerated parallel hierarchical extreme learning machine on flink for Big Data," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 47, no. 10, pp. 2740–2753, Oct. 2017.
- [128] M. M. Rathore, H. Son, A. Ahmad, A. Paul, and G. Jeon, "Real-time Big Data stream processing using GPU with spark over hadoop ecosystem," *Int. J. Parallel Program.*, vol. 46, no. 3, pp. 630–646, Jun. 2018.
- [129] Z. Zhang, Z. Liu, Q. Jiang, J. Chen, and H. An, "RDMA-based apache storm for high-performance stream data processing," *Int. J. Parallel Program.*, vol. 49, no. 5, pp. 671–684, Oct. 2021.
- [130] V. E. Venugopal, M. Theobald, D. Tassetti, S. Chaychi, and A. Tawakuli, "Targeting a light-weight and multi-channel approach for distributed stream processing," *J. Parallel Distrib. Comput.*, vol. 167, pp. 77–96, 2022.
- [131] J. López-Gómez, J. Fernández Muñoz, D. del Rio Astorga, M. F. Dolz, and J. D. Garcia, "Exploring stream parallel patterns in distributed MPI environments," *Parallel Comput.*, vol. 84, pp. 24–36, 2019.
- [132] H. Yviquel and G. Araújo, "The cloud as an openmp offloading device," in *Proc. 46th Int. Conf. Parallel Process.*, 2017, pp. 352–361.
- [133] H. Yviquel, L. Cruz, and G. Araujo, "Cluster programming using the openmp accelerator model," *ACM Trans. Archit. Code Optim.*, vol. 15, no. 3, Aug. 2018.
- [134] M. Mahdi Javanmard, Z. Ahmad, J. Zola, L.-N. Pouchet, R. Chowdhury, and R. Harrison, "Efficient execution of dynamic programming algorithms on apache spark," in *Proc. 2020 IEEE Int. Conf. Cluster Comput.*, 2020, pp. 337–348.



Mengbing Zhou received the bachelor's degree in information and computing science from the South China University of Technology, in 2022. He is currently working toward the master's degree in computer technology with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, with research interests in cloud computing and Big Data analytics.



Qiuyan Li received the graduated degree from the University of Science and Technology of China and the master's degree in software engineering, in 2023. She began her studies, in 2022 at the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, focusing on the integration of parallel computing and Big Data.



Mingyuan Cai received the graduated degree from the University of Chinese Academy of Sciences and the master's degree in computer technology, in 2024. He began his studies, in 2021 with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, focusing on the integration of distributed system and work flow scheduling.



Chengzhong Xu (Fellow, IEEE) received the PhD degree from the University of Hong Kong, in 1993. He is currently the dean of Faculty of Science and Technology, University of Macau, China, and the director of the Institute of Advanced Computing and Data Engineering, Shenzhen Institutes of Advanced Technology of Chinese Academy of Sciences. His research interest includes parallel and distributed systems, service and cloud computing, and software engineering. He has published more than 200 papers in journals and conferences. He serves on a number of journal editorial boards, including *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Cloud Computing*, *Journal of Parallel and Distributed Computing*, and *China Science Information Sciences*.



Yang Wang (Member, IEEE) received the BSc degree in applied mathematics from the Ocean University of China, Qingdao, China, in 1989, the MSc degree in computer science from Carleton University, Ottawa, Canada, in 2001, and the PhD degree in computer science from the University of Alberta, Edmonton, Canada, in 2008. He currently works with the Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, as a professor. His research interests include cloud computing, Big Data analytics, and Java virtual machines on multicores. He is an Alberta Industry R&D associate (2009–2011), and a Canadian Fulbright Scholar (2014–2015).